Design using System Verilog

Assignment_4(Assignment_3 written in classroom)

- Toshiv Kumar, 1024060266, 3F32, ECE

-

Aim: Design and simulation of a sequence detector which has one input and one output. If the input sequence is 1011, then the output of the circuit is ‘1’.

Perform the following activities:

1.Write the System Verilog Code for a mealy FSM based sequence detector with overlapping allowed to detect sequence: “1011”. Verify its operation.

Code:

Design file:

```
`timescale 1ns / 1ps


// Toshiv Kumar(NAME) , 1024060266(Roll Number)


module question_1_mealy(
    input  logic clk,
    input  logic rst,
    input  logic x,
    output logic z
);


    typedef enum logic [1:0] {
        s0,
        s1,
        s2,
        s3
```

```
} state_t;

// enum syntax:
// typedef enum logic [n-1:0] {name1, name2, name3} type_name;
// this creates a state type with fixed bit width
// in hardware, it becomes a set of flip-flops storing the current state encoding

state_t present_state, next_state;

always_ff @(posedge clk or posedge rst) begin
    if (rst)
        present_state <= s0;
    else
        present_state <= next_state;
end

always_comb begin
    z = 1'b0;
    next_state = present_state;

    unique case (present_state)
        s0: begin
            if (x) next_state = s1;
            else   next_state = s0;
        end

        s1: begin
            if (x) next_state = s1;
            else   next_state = s2;
```

```
        end

        s2: begin
            if (x) next_state = s3;
            else   next_state = s0;
        end

        s3: begin
            if (x) begin
                z = 1'b1;
                next_state = s1;
            end
            else begin
                next_state = s2;
            end
        end
      endcase
   end

endmodule
```

TB file:

```
`timescale 1ns / 1ps
// Toshiv Kumar(NAME) , 1024060266(Roll Number)

module question_1_TB;
   logic clk;
   logic rst;
   logic x;
```

```
logic z;

question_1_mealy dut(
    .clk(clk),
    .rst(rst),
    .x(x),
    .z(z)
);

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    rst = 1;
    x = 0;
    #12 rst = 0;

    #10 x = 1;
    #10 x = 0;
    #10 x = 1;
    #10 x = 1;

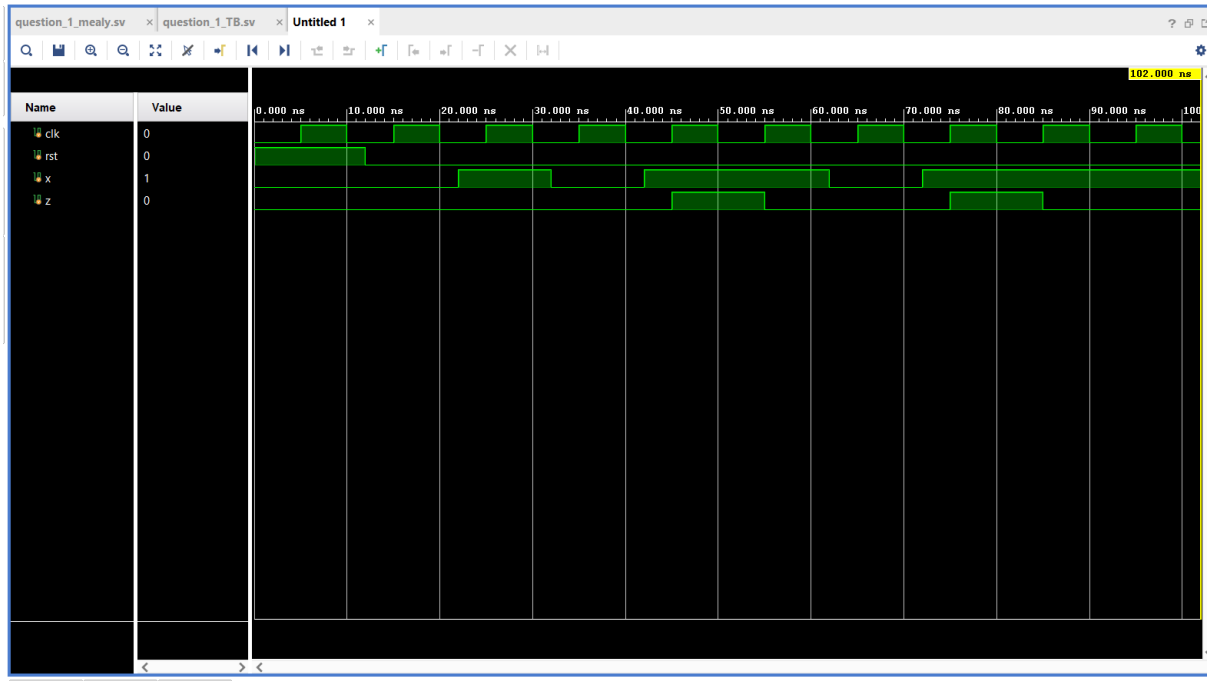
    #10 x = 0;
    #10 x = 1;
    #10 x = 1;

    #20 $finish;
```

```
    end

    initial begin
        $monitor("time=%0t rst=%b x=%b z=%b", $time, rst, x, z);
    end
endmodule
```

Output/Screenshot:



2. Write the System Verilog Code for a Moore FSM-based sequence detector with overlapping allowed to detect the sequence: “1011”. Verify its operation.

Code:

Design file:

```
`timescale 1ns / 1ps

// Toshiv Kumar(NAME) , 1024060266(Roll Number)
```

```
module question_2_moore(
    input  logic clk,
    input  logic rst,
    input  logic x,
    output logic z
);

    typedef enum logic [2:0] {
        s0,
        s1,
        s2,
        s3,
        s4
    } state_t;

    // enum syntax:
    // typedef enum logic [n-1:0] {s0, s1, s2, ...} state_t;
    // this creates a state type with encoded values
    // in hardware, it is stored in flip-flops as the present state

    state_t present_state, next_state;

    always_ff @(posedge clk or posedge rst) begin
        if (rst)
            present_state <= s0;
        else
            present_state <= next_state;
    end
```

```
always_comb begin
    next_state = present_state;

    unique case (present_state)
        s0: begin
            if (x) next_state = s1;
            else   next_state = s0;
        end

        s1: begin
            if (x) next_state = s1;
            else   next_state = s2;
        end

        s2: begin
            if (x) next_state = s3;
            else   next_state = s0;
        end

        s3: begin
            if (x) next_state = s4;
            else   next_state = s2;
        end

        s4: begin
            if (x) next_state = s1;
            else   next_state = s2;
        end
    endcase
```

```
    end

    assign z = (present_state == s4);

endmodule
```

TB:

```
`timescale 1ns / 1ps

// Toshiv Kumar(NAME) , 1024060266(Roll Number)

module question_2_TB;
    logic clk;
    logic rst;
    logic x;
    logic z;

    question_2_moore dut(
        .clk(clk),
        .rst(rst),
        .x(x),
        .z(z)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
```

```
    end

    initial begin
        rst = 1;
        x = 0;
        #12 rst = 0;

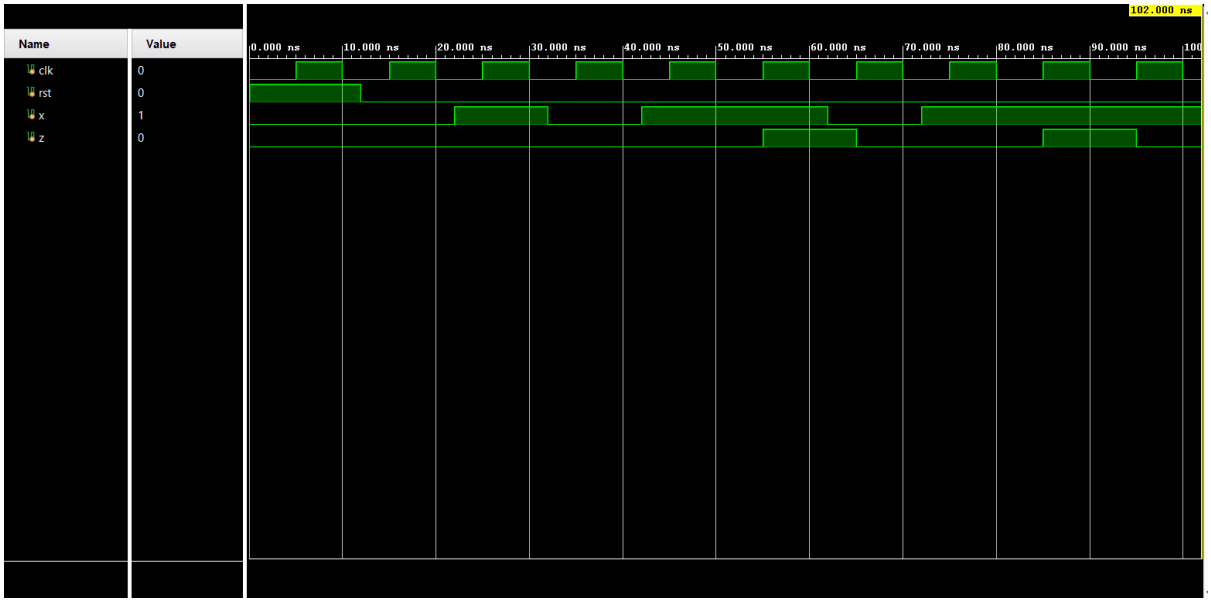
        #10 x = 1;
        #10 x = 0;
        #10 x = 1;
        #10 x = 1;

        #10 x = 0;
        #10 x = 1;
        #10 x = 1;

        #20 $finish;
    end

    initial begin
        $monitor("time=%0t rst=%b x=%b z=%b", $time, rst, x, z);
    end
endmodule
```

Output/Screenshot:



Learnings from this Experiment:

There are many things that I learned from this experiment.

I learned how to use the enum datatype to store states**. I learned that if nothing is mentioned in the question about overlapping allowed or not, then assume that overlapping is not allowed.** I learned about Mealy and Moore FSMs and the optimized implementation of FSMs in System Verilog. I learned about modern keywords such as typedef, always_ff, always_comb, etc.
Thank you for the assignment.